

PROJECT REPORT

POCKETSMART AI

Your Smart Budget & Recommendation Assistant

A Generative AI Based Budget Planning and Recommendation System

SRINIVASAN COLLEGE OF ARTS AND SCIENCES

BHARATHIDASAN UNIVERSITY

SUBMITTED BY

BASKARAN G

Register Number: 25051750500111009

PROJECT TEAM ID

SWTID-2026-5159

ACADEMIC YEAR

2026 – 2027

INTRODUCTION

PocketSmart AI is a smart, AI-powered budget and recommendation assistant designed to help users plan and manage their spending across different lifestyle needs.

The system provides personalized, budget-aware recommendations for home interior planning, party planning, and jewelry selection.

It uses user preferences, budgets, contextual details, and optional images to generate suitable suggestions.

By combining FastAPI with Gemini 1.5 Flash Pro and integrating information from platforms such as Amazon, Flipkart, IKEA, Swiggy, Zomato, and OYO, PocketSmart AI aims to make planning easier, faster, and more convenient while helping users stay within their defined budgets.



PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Description

Managing budgets across different life needs—like home decor, event planning, or jewelry shopping—can be overwhelming due to the wide variety of products, platforms, and price ranges. **PocketSmart AI** addresses this challenge through a **GenAI-powered, cross-platform recommendation system** that delivers personalized, budget-based suggestions for products and services. Using **Gemini 1.5 Flash Pro**, PocketSmart AI analyzes user preferences, budgets, and contextual needs to generate curated recommendations from popular platforms such as **Amazon, Flipkart, IKEA, Swiggy, Zomato, OYO**, and more. The platform helps users plan interior designs, parties, and even select matching jewelry for special occasions—all within their defined budgets. PocketSmart AI transforms budgeting into an intelligent, user-friendly experience through its smart planners, adaptive forms, and recommendation engine that bridges multiple categories and e-commerce ecosystems.

Scenario 1: Home Interior Planning with Smart Budget Allocation

Users begin by selecting the **Home Budget Planner**. After entering their budget, they specify room types (e.g., Living Room, Kitchen, Bedroom) and quantities for lights, ceiling fans, dining tables, etc. Gemini 1.5 Flash Pro processes this data and recommends cost-effective options for each category across platforms like IKEA and Amazon. Recommendations are balanced across functionality, style, and price, ensuring users can decorate their spaces beautifully without overspending.

Scenario 2: AI-Based Party Budget Planning

In the **Party Planner**, users input their total budget, guest count, event type, and venue details. Based on these inputs, the AI allocates the budget proportionally across **catering, decoration, and entertainment**, sourcing options from vendors listed on Swiggy, Zomato, and even accommodation services like OYO. The system tailors suggestions based on event type (e.g., birthday, corporate, wedding), helping users organize a successful event without financial guesswork.

Scenario 3: Jewelry Recommendations for Occasions

The **Jewelry Budget Planner** allows users to enter a budget, select the occasion, and define style preferences. Users can optionally upload an outfit image for the AI to consider color coordination and aesthetics. Gemini 1.5 Flash Pro analyzes the data and offers jewelry options from platforms like Amazon and Flipkart, tailored to occasion and style. This ensures that users receive elegant, matching jewelry suggestions that fit both their personal style and financial plan.

Architecture Overview

PocketSmart AI is an intelligent, modular platform designed to generate personalized, budget-aware recommendations across domains like home interiors, party planning, and jewelry selection. It integrates a lightweight Flask backend with Google’s **Gemini 1.5 Flash Pro**, a powerful multimodal AI model that processes user inputs (text, numbers, and images) to deliver smart recommendations.

Core Technologies

- **Flask:** Backend service layer for request handling, routing, and session control.
- **Gemini 1.5 Flash Pro:** AI model used for understanding budget context, analyzing images, generating structured suggestions, and recommending platform-specific products.
- **Third-party APIs (Amazon, Flipkart, IKEA, Zomato, etc.):** Product and service data sourcing.
- **Frontend (HTML/CSS + JS):** Interactive user interface for input collection and AI output display.

Component-Wise Architecture

Component	Description
Frontend UI	Web interface built with HTML/CSS allows users to input budgets, preferences, and images. Displays tailored AI-driven recommendations for each category.
Flask Backend	Manages user sessions, routes (/generate-home, /generate-party, /generate-jewelry), and handles communication between UI and Gemini AI.
Gemini AI Layer	Processes inputs using Gemini 1.5 Flash Pro to:



Pre-requisites

1. IBM Cloud Account Setup: [IBM Cloud Console](#)
2. Watsonx Access and Configuration: [Watsonx Overview](#)
3. Python Basics: [Python.org](#)
4. FastAPI Framework: [FastAPI Docs](#)

Project Workflow

1. Gemini AI Setup & Initialization

- **Activity 1.1:** Set up Google Cloud access (or platform) with Gemini 1.5 Flash Pro API access.
- **Activity 1.2:** Configure API keys, usage limits, and test integration for multimodal support (text + image).
- **Activity 1.3:** Fine-tune prompts for budget interpretation and product recommendation generation.

2. Core Functionality Development

- **Activity 2.1:** Design Flask-based backend structure for routing, session control, and planner modules.
- **Activity 2.2:** Create endpoints for Home Planner, Party Planner, and Jewelry Planner.
- **Activity 2.3:** Integrate Gemini prompts and templates for each planner with support for text + optional image inputs.

3. Backend Implementation (app.py)

- **Activity 3.1:** Define Flask routes like `/generate-home`, `/generate-party`, `/generate-jewelry`.
- **Activity 3.2:** Organize backend using modular structure:
 - `routes/`: Endpoints
 - `services/`: Recommendation logic & AI calls
 - `models/`: Input/output schemas
- **Activity 3.3:** Setup CORS headers for frontend communication and implement basic session management.
- **Activity 3.4:** Include mock API calls or simulated scraping from Amazon, IKEA, Zomato, etc.



- **Activity 4.1:** Build responsive HTML forms for:
 - Home Interior Planner (room details, quantities)
 - Party Planner (event type, budget, guests)
 - Jewelry Planner (budget, occasion, outfit image upload)
- **Activity 4.2:** Add dynamic result display and formatting for recommendations using JavaScript.

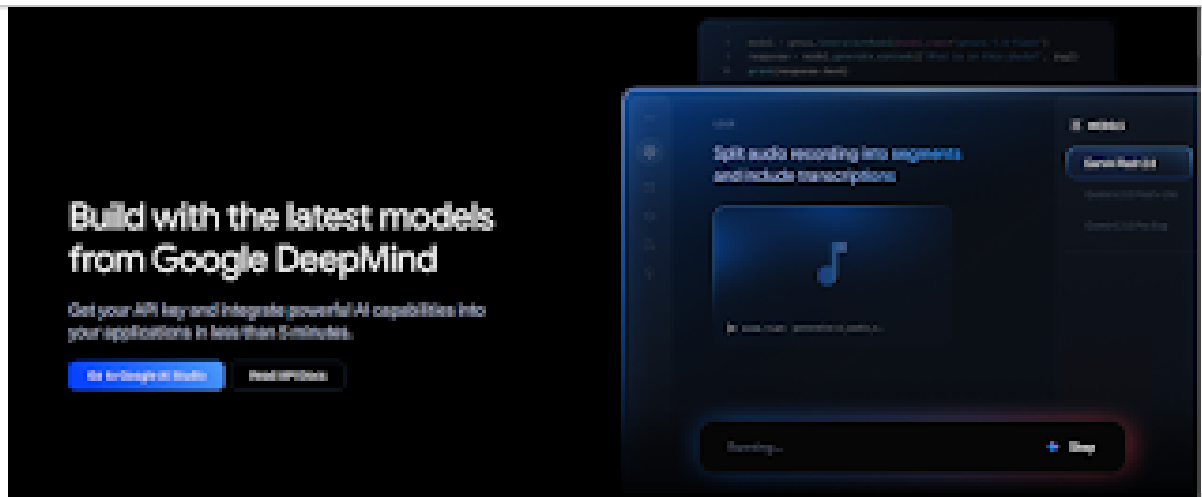
5. Testing & Optimization

- **Activity 5.1:** Perform testing with real-world use cases across all three planners.
- **Activity 5.2:** Evaluate Gemini response quality, budget adherence, and platform accuracy.
- **Activity 5.3:** Optimize prompts and add validations to prevent input edge-case failures.
- **Activity 5.4:** Add fallback/default recommendations when AI returns insufficient results.

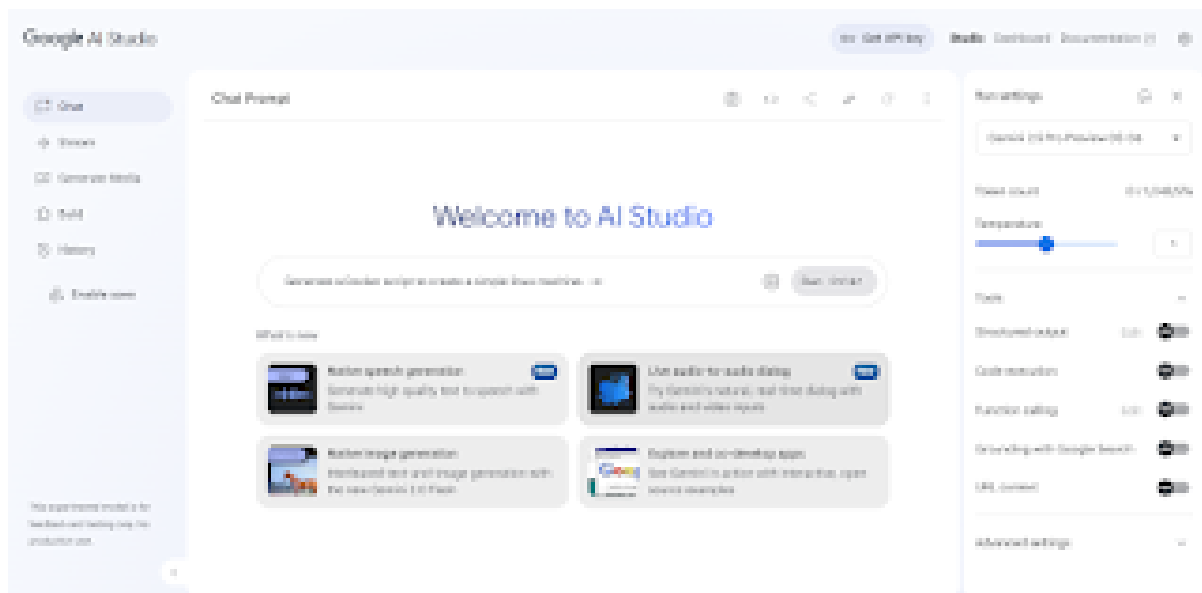
MILESTONE 1: Gemini AI Initialization

In this foundational stage, the objective is to establish and configure the core AI infrastructure required to power **PocketSmart AI**. The platform relies on **Gemini 1.5 Flash Pro**—Google's multimodal foundation model—to interpret user inputs, process budgets, and generate personalized recommendations. This milestone ensures that the development environment is equipped with authenticated access to Gemini's APIs and is capable of performing both text-based reasoning and image analysis (specifically for the Jewelry Planner). Proper completion of this setup is essential, as it enables all AI-driven functionalities like budget understanding, category-wise recommendations, and multimodal analysis to operate seamlessly.

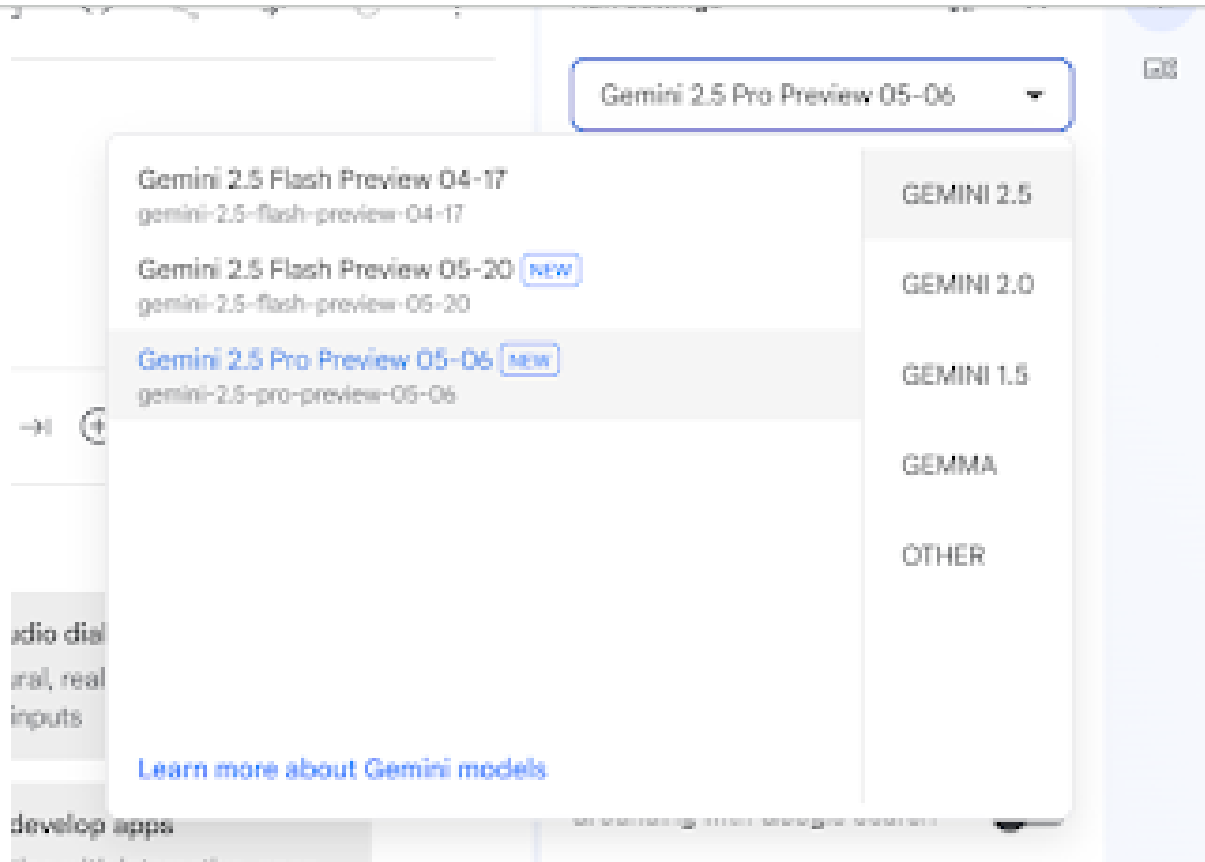
- **Activity 1.1: Set up Google Cloud account and enable Gemini API**
 - Sign in or create a Google Cloud account via console.cloud.google.com.



- Accept all terms and conditions and click on Get API key



- You will get trending models here you can select the model here or you can set the model by go to the API Documentation.



Activity 1.2: Configure access and permissions

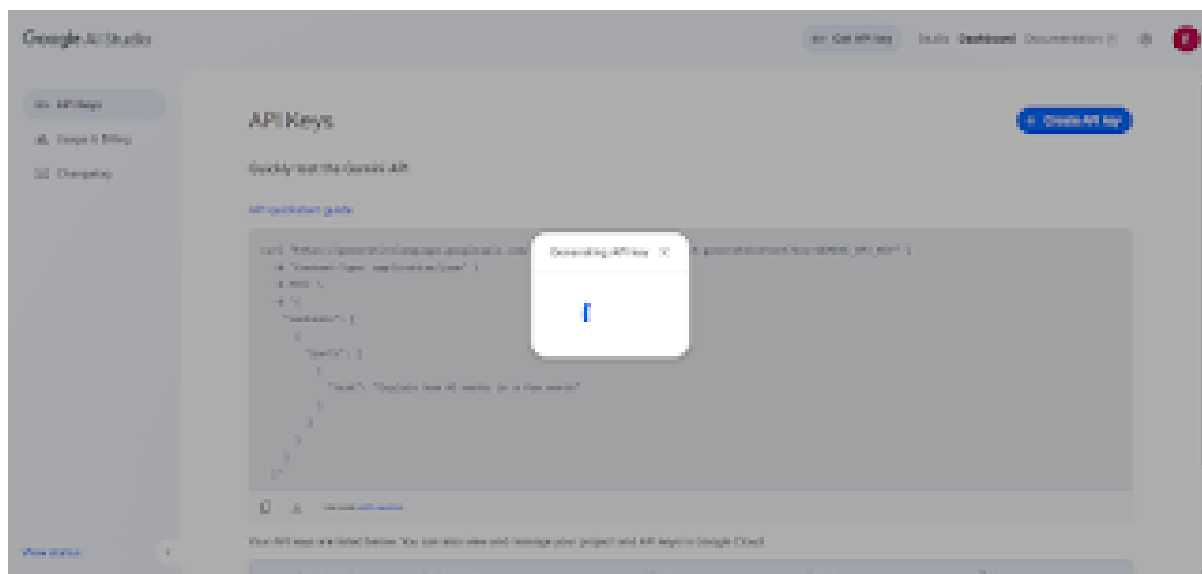
- You will get interface like this and now click on Create API key



PocketSmart AI_ Your Smart Budget & Recommendation Assistant ...

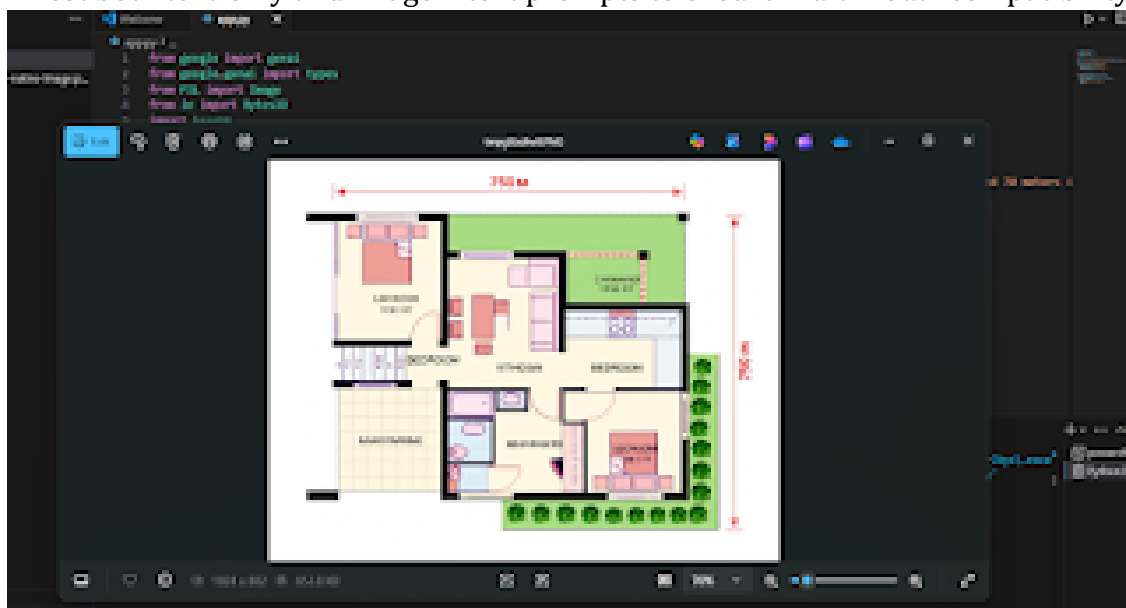


- Once click on API key you will the api key and make sure to copy the API key



```
1 from google import genai
2 from google.genai import types
3 from PIL import Image
4 from io import BytesIO
5 import asyncio
6
7 client = genai.Client(api_key="sk-1234567890abcdef")
8
9 prompt = ["Generate a detailed 3D architectural floor plan for a modern two-story house with a total area of 150 meters in length and 70 meters in width. The plan should include a living room, kitchen, dining area, bedrooms, bathrooms, and a garden. The style should be modern and minimalist."
10
11 response = client.models.generate_content(
12     model="gemini-1.5-flash-exp:latest-generation",
13     contents=prompt,
14     config=types.GenerateContentConfig(
15         response_modalities=["text", "image"]
16     )
17 )
18
19 for part in response.candidates[0].content.parts:
20     if part.text is not None:
21         print(part.text)
22     elif part.inline_data is not None:
23         image = Image.open(BytesIO(part.inline_data.data))
24         image.save("gemini-output-image.png")
25         image.show()
```

- Test both text-only and image + text prompts to ensure multimodal compatibility.

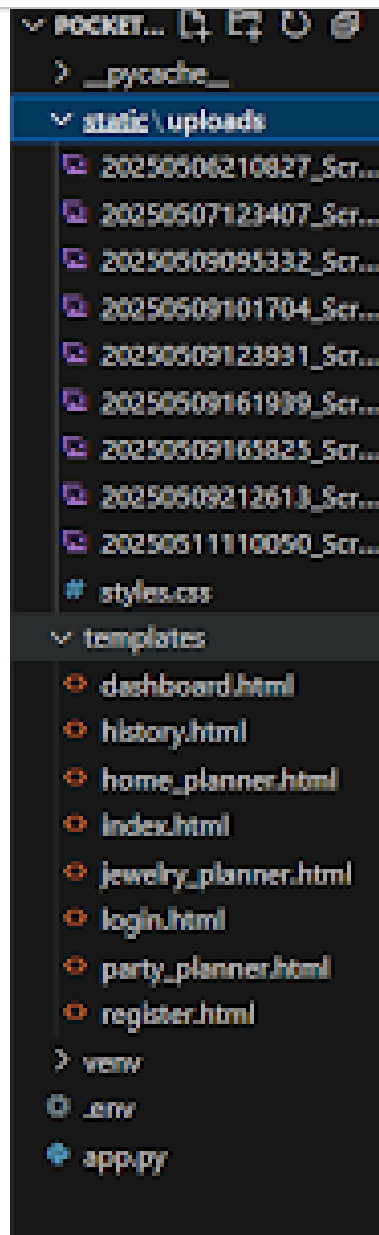




MILESTONE 2: Core Functionalities Development

This milestone focuses on building the **core backend intelligence** of **PocketSmart AI** using the Flask framework and Google's **Gemini 1.5 Flash Pro** foundation model. The primary goal is to develop a robust and modular system capable of accepting user budgets (and optional image inputs), intelligently interpreting the context, and returning domain-specific, platform-aware recommendations. The backend handles prompt generation, API communication with Gemini, modular planning logic (for Home, Party, and Jewelry), and optional fallback strategies when product matches are limited. All core logic is abstracted into a utility service file ([gemini_utils.py](#)) that handles prompt orchestration, budget formatting, domain segmentation, and image analysis for multimodal tasks.

File Explorer Structure



Activity 2.1: Fast API Instilization and import libraries

- Import all required libraries for the project



```
import os
import base64
import shutil
import urllib.parse
from fastapi import FastAPI, HTTPException, Depends, File, UploadFile, Form, Request, status, Cookie
from fastapi.responses import JSONResponse, RedirectResponse
from fastapi.middleware.cors import CORSMiddleware
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from fastapi.responses import HTMLResponse
from typing import List, Optional, Dict, Any, Union
from pydantic import BaseModel, EmailStr
from passlib.context import CryptContext
from jose import JWTError, jwt
from datetime import datetime, timedelta
from PIL import Image
from io import BytesIO
import google.generativeai as genai
from dotenv import load_dotenv
import asyncio
from typing import Dict, Any, List, Optional
import uuid
```

- Fast API Initialization

```
# FastAPI app initialization
app = FastAPI(title="PocketSmart: AI Budget Planner")

SECRET_KEY = os.getenv("SECRET_KEY", "your_secret_key")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30
```



```
# Load environment variables
load_dotenv()

# Configure google serial api - in this case, with consistent environment variable name
API_KEY = os.getenv("GOOGLE_API_KEY")
if not API_KEY:
    # try alternative environment variable name
    API_KEY = os.getenv("SERIAL_API_KEY")

if not API_KEY:
    raise ValueError("no google API key found in environment variables. Please set GOOGLE_API_KEY in your .env file.")

gmail.configure(api_key=API_KEY)

# Password hashing context
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
authz_scheme = CustomPasswordChecker(token="token", auto_error=False) # allow optional token

class RegisterForm(BaseModel):
    username: str
    email: str
    full_name: Optional[str] = None
    password: str

# Configure CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # adjust in production
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Static files and templates
templates = Jinja2Templates(directory="templates")
os.makedirs("static/uploads", exist_ok=True)
app.mount("/static", StaticFiles(directory="static"), name="static")
```

Activity 2.2: Main Functionality code routings

- `/generate-home`: Generates home interior recommendations (furniture, decor, lighting)

```
@app.get("/home-planner", response_class=HTMLResponse)
async def home_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """home budget planner page"""
    return templates.TemplateResponse("home_planner.html", {"request": request, "user": current_user})
```

- get home_recommendations by using the function



```
prompt = f"""
I need interior design product recommendations for a home in India with a total budget of ₹{budget_input.total_budget:.2f}.
Requirements:
- {budget_input.num_lights} lights/lighting fixtures
- {budget_input.num_fans} ceiling fans
- {budget_input.num_furniture} furniture pieces
- {budget_input.num_dining_tables} dining tables

Additional rooms to consider:
{["- living room" if budget_input.has_living_room else ""],
  ["- kitchen" if budget_input.has_kitchen else ""],
  ["- bedroom" if budget_input.has_bedroom else ""]}

Additional requirements: {budget_input.additional_requirements or "None"}

Please provide a detailed budget breakdown with product recommendations "available in India".
Use "Indian brands and pricing". Include "search terms" suitable for Indian shopping platforms.

Format your response as JSON with the following structure:
{
  "total_budget": {budget_input.total_budget:.2f},
  "budget_breakdown": [
    {
      "category": "lighting",
      "allocation": 0.0,
      "items": [
        {
          "name": "",
          "description": "",
          "estimated_price": 0.0,
          "quantity": 0,
          "search_terms": ""
        }
      ]
    }
  ]
}
```

- linking the websites like amazon,flipkar,ikea ect.... to get the products

```
def get_home_recommendations(budget_input: HomeBudgetInput) -> dict:
    """
    Generate home recommendations based on budget and requirements.
    """
    # Generate recommendations from the model
    response = model.generate_content(prompt)
    result = extract_json_from_response(response.text)

    # Add shopping links for each item
    for category in result.get("budget_breakdown", []):
        for item in category.get("items", []):
            search_terms = item.get("search_terms", "")
            if search_terms:
                item["shopping_links"] = [
                    "amazon": f"https://www.amazon.in/s?k={urllib.parse_qs.parse_qs(search_terms)}",
                    "flipkart": f"https://www.flipkart.com/search?q={urllib.parse_qs.parse_qs(search_terms)}",
                    "ikea": f"https://www.ikea.com/in/en/search/?q={urllib.parse_qs.parse_qs(search_terms)}",
                    "ajio": f"https://www.ajio.com/search/?q={urllib.parse_qs.parse_qs(search_terms)}",
                    "myntra": f"https://www.myntra.com/search?q={urllib.parse_qs.parse_qs(search_terms)}",
                    "ajio": f"https://www.ajio.com/search/?q={urllib.parse_qs.parse_qs(search_terms)}"
                ]

    return result
```



```
def usd_to_inr(amount_usd: float, exchange_rate: float = 83.0) -> float:
    """Convert USD amount to INR using the specified exchange rate"""
    return amount_usd * exchange_rate
```

- `/generate-party`: Suggests catering, venue, and decoration options for events

```
@app.get("/party-planner", response_class=HTMLResponse)
async def party_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """Party budget planner page"""
    return templates.TemplateResponse("party_planner.html", {"request": request, "user": current_user})
```

- get home_recommendations by using the function

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
    """Generate party planning recommendations within budget in INR for Indian market"""
    try:
        # Create prompt for AI model
        prompt = f"""
        I need party planning recommendations for India with a total budget of ₹{budget_input.total_budget:.2f}.

        Party details:
        - Type: {budget_input.party_type}
        - Number of guests: {budget_input.num_guests}
        - Venue type: {budget_input.venue_type or "Not specified"}
        - Catering needed: {"Yes" if budget_input.needs_catering else "No"}
        - Decoration needed: {"Yes" if budget_input.needs_decoration else "No"}
        - Entertainment needed: {"Yes" if budget_input.needs_entertainment else "No"}

        Additional requirements: {budget_input.additional_requirements or "None"}

        Please provide a detailed budget breakdown with specific recommendations available in India using INR prices.
        Use Indian brands, services, and typical cost expectations.

        Format your response as JSON with the following structure:
        {{
            "total_budget": {budget_input.total_budget:.2f},
            "budget_breakdown": [
                {{
                    "category": "venue",
                    "allocation": 0.0,
                    "items": [
                        {{
                            "name": "",
                            "description": "",
                            "estimated_price": 0.0,
                            "quantity": 0,
                            "search_terms": ""
                        }}
                    ]
                }}
            ]
        }}
```

```

        "verse_suggestions": [
            {
                "name": "",
                "type": "",
                "capacity": 0,
                "estimated_cost": 0.0,
                "search_terms": ""
            }
        ],
        "remaining_budget": 0.0,
        "additional_suggestions": []
    })

    Ensure all costs are in INR and total does not exceed the given budget.
    Provide search terms suitable for Indian websites such as Bocketyshow, Seiggy, Flipkart, etc.
    ...

    response = model.generate_content(prompt)
    result = extract_json_from_response(response.text)

    # Create INR calculation table
    result["calculation_table_inr"] = []
    categories = {}

    for category in result.get("budget_breakdown", []):
        cat_name = category.get("category", "Misc")
        cat_allocation = category.get("allocation", 0)

        if cat_name not in categories:
            categories[cat_name] = {
                "category": cat_name,
                "items_count": 0,
                "total_cost": 0,
                "percentage_of_budget": 0
            }

```

- linking the websites like amazon,flipkar,ikea ect.... to get the products



```

    categories[cat_name]["percentage_of_budget"] = (categories[cat_name]["total_cost"] / result["total_budget"]) * 100

for cat_data in categories.values():
    result["calculation_table_for"].append(cat_data)

# define relevant shopping platforms for each category
category_platforms = {
    "house": ["google", "booking", "lastminute", "expedia", "trivago"],
    "clothing": ["wagdy", "mango"],
    "food": ["wagdy", "mango", "bigbasket", "mama", "flipkart"],
    "drinks": ["wagdy", "mango", "bigbasket", "mama", "flipkart"],
    "decoration": ["mama", "flipkart", "mango", "myntra"],
    "entertainment": ["bookmyshow", "mama", "flipkart"],
    "gifts": ["mama", "flipkart", "myntra", "mango"],
    "photography": ["google", "mama", "flipkart"],
    "music": ["mama", "flipkart", "bookmyshow"],
    "games": ["mama", "flipkart"],
    "accessories": ["mama", "flipkart", "myntra", "mango"],
    "transportation": ["lastminute", "google"],
    "return_gifts": ["mama", "flipkart", "myntra", "mango"]
}

# default platforms if category is not in our predefined list
default_platforms = ["mama", "flipkart", "google"]

# add shopping links for each item
for category in result.get("budget_breakdown", []):
    cat_name = category.get("category", "").lower()
    relevant_platforms = category_platforms.get(cat_name, default_platforms)

    for item in category.get("items", []):
```

- using the if condition to categorize the situation based products

```
def get_party_recommendations(budget_input, partyhelpinput) -> dict:
    for category in result.get("budget_breakdown", []):
        cat_name = category.get("category", "").lower()
        relevant_platforms = category_platforms.get(cat_name, default_platforms)

        for item in category.get("items", []):
            search_terms = item.get("search_terms", "")
            if search_terms:
                item["shopping_links"] = {}

                # only add relevant platforms for this category
                if "mama" in relevant_platforms:
                    item["shopping_links"]["mama"] = f"https://www.mama.in/s?h={urlib.parse.quote_plus(search_terms)}"
                if "flipkart" in relevant_platforms:
                    item["shopping_links"]["flipkart"] = f"https://www.flipkart.com/search?q={urlib.parse.quote_plus(search_terms)}"
                if "bigbasket" in relevant_platforms:
                    item["shopping_links"]["bigbasket"] = f"https://www.bigbasket.com/search/?q={urlib.parse.quote_plus(search_terms)}"
                if "wagdy" in relevant_platforms:
                    item["shopping_links"]["wagdy"] = f"https://www.wagdy.com/search?q={urlib.parse.quote_plus(search_terms)}"
                if "mango" in relevant_platforms:
                    item["shopping_links"]["mango"] = f"https://www.mango.com/search?q={urlib.parse.quote_plus(search_terms)}"
                if "bookmyshow" in relevant_platforms:
                    item["shopping_links"]["bookmyshow"] = f"https://in.bookmyshow.com/search?q={urlib.parse.quote_plus(search_terms)}"
                if "myntra" in relevant_platforms:
                    item["shopping_links"]["myntra"] = f"https://www.myntra.com/search?q={urlib.parse.quote_plus(search_terms)}"
                if "mango" in relevant_platforms:
                    item["shopping_links"]["mango"] = f"https://www.mango.com/search?q={urlib.parse.quote_plus(search_terms)}"
                if "google" in relevant_platforms:
                    item["shopping_links"]["google"] = f"https://www.google.com/search?q={urlib.parse.quote_plus(search_terms)}"
                if "booking" in relevant_platforms:
                    item["shopping_links"]["booking"] = f"https://www.booking.com/search.html?ss={urlib.parse.quote_plus(search_terms)}"
                if "lastminute" in relevant_platforms:
                    item["shopping_links"]["lastminute"] = f"https://www.lastminute.com/hotels/india/{urlib.parse.quote_plus(search_terms)}"
                if "expedia" in relevant_platforms:
                    item["shopping_links"]["expedia"] = f"https://www.expedia.com/search/Hotels-in-{urlib.parse.quote_plus(search_terms)}"
                if "trivago" in relevant_platforms:
                    item["shopping_links"]["trivago"] = f"https://www.trivago.in/property/{urlib.parse.quote_plus(search_terms)}"
                if "mango" in relevant_platforms:
                    item["shopping_links"]["mango"] = f"https://www.mango.com/search?q={urlib.parse.quote_plus(search_terms)}"
```



```

search_terms = venue.get('search_terms', '')
if search_terms:
    venue['search_links'] = {}
    if "google" in venue_platforms:
        venue['search_links']['google'] = f"https://www.google.com/search?q={urlib.parse.quote_plus(search_terms)}"
    if "booking" in venue_platforms:
        venue['search_links']['booking'] = f"https://www.booking.com/search.html?q={urlib.parse.quote_plus(search_terms)}"
    if "hotels.com" in venue_platforms:
        venue['search_links']['hotels.com'] = f"https://www.hotels.com/deals/hotels-listing/hotels/{urlib.parse.quote_plus(search_terms)}"
    if "opendesktop" in venue_platforms:
        venue['search_links']['opendesktop'] = f"https://www.opendesktop.com/search/?location={urlib.parse.quote_plus(search_terms)}"
    if "advertiser" in venue_platforms:
        venue['search_links']['advertiser'] = f"https://www.advertiser.in/property/search?searchTerm={urlib.parse.quote_plus(search_terms)}"

return result

except Exception as e:
    raise ValueError(f"Error generating recommendations: {str(e)}")

```

- `/generate-jewelry`: Accepts text + image inputs to return style-matched jewelry options.
- `async def jewelry_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):`
 - `"""Jewelry budget planner page"""`
 - `return templates.TemplateResponse("jewelry_planner.html", {"request": request, "user": current_user})`

- get home_recommendations by using the function

```

def get_jewelry_recommendations(budget_input: FormInput, image_path: Optional[str] = None) -> dict:
    """Generate jewelry recommendations based on uploaded dress and budget in INR (India-specific)"""
    try:
        # Base prompt with INR context
        base_prompt = f"""
        I need jewelry recommendations for India with a total budget of ₹{budget_input.total_budget}.

        Occasions: {budget_input.occasions}
        Preferences: {budget_input.preferences} or "Not specified"
        Provide only India-relevant styles, availability, and price ranges in INR.
        """

        # Include image-based outfit analysis
        if image_path:
            img = Image.open(image_path)
            prompt = base_prompt + """

            An image of the outfit is uploaded. Suggest jewelry that complements it, considering color, design, and occasion appropriateness.

            Format the output as JSON:
            {
                "outfit_analysis": {
                    "color": [],
                    "style": "",
                    "occasion": ""
                },
                "total_budget": ₹0,
                "jewelry_recommendations": [
                    {
                        "item_type": "",
                        "description": "",
                        "style": "",
                        "estimated_price": ₹0,
                        "search_terms": ""
                    }
                ]
            }
            """
            # Generate JSON output

```



```
Make sure prices are in INR and stay within budget.
Include Indian-friendly search terms for shopping.
"""
response = model.generate_content([prompt, img])
else:
    # Only text input
    prompt = base_prompt + """

    Format the output as JSON:
    {
        "total_budget": 0.0,
        "jewelry_recommendations": [
            {
                "item_type": "",
                "description": "",
                "style": "",
                "estimated_price": 0.0,
                "search_terms": ""
            }
        ],
        "remaining_budget": 0.0,
        "styling_tips": []
    }

    Keep prices in INR and relevant to Indian brands.
    """
    response = model.generate_content(prompt)

result = extract_json_from_response(response.text)

# Add shopping links for each item (India-specific)
for item in result.get("jewelry_recommendations", []):
    search_terms = item.get("search_terms", "")
```

- linking the websites like amazon,flipkar,ikea ect.... to get the products

```
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
    # Add shopping links for each item (India-specific)
    for item in result.get("jewelry_recommendations", []):
        search_terms = item.get("search_terms", "")
        if search_terms:
            item["shopping_links"] = {
                "amazon": f"https://www.amazon.in/s?k={urlib.parse.quote_plus(search_terms)}",
                "Flipkart": f"https://www.flipkart.com/search?q={urlib.parse.quote_plus(search_terms)}",
                "bluestone": f"https://www.bluestone.com/search.html?query={urlib.parse.quote_plus(search_terms)}",
                "tanishq": f"https://www.tanishq.co.in/search?q={urlib.parse.quote_plus(search_terms)}",
                "caratslane": f"https://www.caratslane.com/search?q={urlib.parse.quote_plus(search_terms)}",
                "malorra": f"https://www.malorra.com/search?q={urlib.parse.quote_plus(search_terms)}",
                "weeda": f"https://www.weeda.com/search?q={urlib.parse.quote_plus(search_terms)}"
            }

    return result

except Exception as e:
    raise HTTPException(500, f"Error generating recommendations: {str(e)}")
```



Activity 2.3: Login and Register page routings

- Login page

/login: Authenticates user credentials and initiates a session upon successful login.

```
@app.get("/login", response_class=HTMLResponse)
async def login_page(request: Request):
    """serve the login page"""
    # Check if user is already logged in
    try:
        token = await get_token(request)
        if token:
            user = await get_current_user(request, token)
            if user:
                return RedirectResponse(url="/dashboard", status_code=status.HTTP_302_FOUND)
    except Exception:
        pass

    return templates.TemplateResponse("login.html", {"request": request})
```

- Register Page

/register: Handles new user registration by accepting and securely storing user credentials.

```
@app.get("/register", response_class=HTMLResponse)
async def register_page(request: Request):
    """serve the registration page"""
    # Check if user is already logged in
    try:
        token = await get_token(request)
        if token:
            user = await get_current_user(request, token)
            if user:
                return RedirectResponse(url="/dashboard", status_code=status.HTTP_302_FOUND)
    except Exception:
        pass

    return templates.TemplateResponse("register.html", {"request": request})
```

- Logout Page

/logout: Terminates the current user session and redirects to the login page.



```
"""Logout user by blacklisting their token and clearing session"""
token = await get_token(request)

if token:
    # Add token to blacklist
    blacklisted_tokens.add(token)

    try:
        # Extract username from token to remove from active sessions
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username = payload.get("sub")
        if username and username in active_sessions:
            del active_sessions[username]
    except JWTError:
        pass

# Use HttpResponseRedirect instead of JSONResponse for proper redirection
response = HttpResponseRedirect(url="/login", status_code=status.HTTP_302_FOUND)
response.delete_cookie(key="access_token")
```

Activity 2.4: Ensure modular code structure

- **/token**: Issues a secure JWT token after successful authentication, enabling authorized access to protected API routes.

```
@app.post("/token", response_model=Token)
async def login_for_access_token(form_data: OAuth2PasswordRequestForm = Depends()):
    """Login endpoint to get access token"""
    user = authenticate_user(user_db, form_data.username, form_data.password)
    if not user:
        raise HTTPException(
            status_code=401,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )

    # Create access token
    access_token_expires = timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = create_access_token(
        data={"sub": user.username}, expires_delta=access_token_expires
    )

    # Create or update session for the user
    existing_user_data = {}
    if user.username in active_sessions:
        existing_user_data = active_sessions[user.username].user_data
        # If there's an old token, blacklist it
        old_token = active_sessions[user.username].token
        blacklisted_tokens.add(old_token)

    active_sessions[user.username] = UserSession(
        username=user.username,
        login_time=datetime.utcnow(),
        last_activity=datetime.utcnow(),
        token=access_token,
        user_data=existing_user_data
    )

    # Return response with cookie
    response = JSONResponse(content={"access_token": access_token, "token_type": "bearer"})

    response.set_cookie(
        key="access_token",
        value=access_token, # Store token directly without Bearer prefix
        httponly=True,
        max_age=ACCESS_TOKEN_EXPIRE_MINUTES * 60,
        samesite="lax"
    )

    return response
```



- **/session-info**: Retrieves metadata about the current user session, such as user ID and login status.
- **/session-data**: Returns detailed session-specific data used for personalization and recommendation tracking.

```
@app.get("/session-info")
async def get_session_info(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """Get current user's session information"""
    if current_user.username in active_sessions:
        session = active_sessions[current_user.username]
        return {
            "username": session.username,
            "login_time": session.login_time,
            "last_activity": session.last_activity,
            "session_duration": (datetime.utcnow() - session.login_time).total_seconds() // 60, # in minutes
            "user_data": session.user_data
        }
    else:
        raise HTTPException(status_code=404, detail="No active session found")

@app.post("/session-data")
async def update_session_data(
    data: Dict[str, Any],
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Update user's session data"""
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data.update(data)
        active_sessions[current_user.username].last_activity = datetime.utcnow()
        return {"message": "Session data updated", "data": active_sessions[current_user.username].user_data}
    else:
        raise HTTPException(status_code=404, detail="No active session found")
# Fix for the /home-budget endpoint to properly handle the form submission
```

This milestone focuses on implementing the FastAPI-powered backend for **PocketSmart AI**. It acts as the bridge between the frontend interface and the Gemini-based AI recommendation engine defined in `gemini_utils.py`. The backend is designed with modular architecture, providing clean and maintainable API endpoints for planners across Home, Party, and Jewelry domains. It also includes features like CORS setup for cross-origin requests, session handling for personalized user experience, and structured input/output models to ensure smooth AI interaction and reliable frontend rendering.

- **Activity 3.1: Define FastAPI Routes**

- Home Planner (`/generate-home`): Accepts home-related preferences and budget to return tailored product suggestions.

```
@app.post("/home-budget")
async def plan_home_budget(
    budget_input: HomeBudgetInput,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate home budget recommendations"""
    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_home_budget"] = {
            "timestamp": datetime.utcnow().isoformat(),
            "budget": budget_input.total_budget,
            "requirements": {
                "lights": budget_input.num_lights,
                "fans": budget_input.num_fans,
                "furniture": budget_input.num_furniture,
                "dining_tables": budget_input.num_dining_tables
            }
        }

    # Get recommendations
    result = get_home_recommendations(budget_input)

    # Save to history
    save_to_history(
        username=current_user.username,
        recommendation_type="home",
        input_data=budget_input.dict(),
        result=result
    )

    return result
```

- Party Planner (`/generate-party`): Processes party details and guest count to recommend venue, food, and decoration items.

```
def generate_party_budget_recommendations(request: Request,
current_user: UserInDB = Depends(get_current_active_user)):
```

```
    """Generate party budget recommendations"""
    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_party_budget"] = {
            "timestamp": datetime.datetime.utcnow().isoformat(),
            "budget": budget_input.total_budget,
            "party_type": budget_input.party_type,
            "guests": budget_input.num_guests
        }

    # Get recommendations
    result = get_party_recommendations(budget_input)

    # Save to history
    save_to_history(
        username=current_user.username,
        recommendation_type="party",
        input_data=budget_input.dict(),
        result=result
    )

    return result
```

- Jewelry Planner (/generate-jewelry): Uses text and optional image inputs to suggest jewelry based on outfit and occasion.



```
@app.route('/plan_jewelry_budget')
def plan_jewelry_budget():
    total_budget: float = Form(...),
    occasion: str = Form(...),
    preferences: Optional[str] = Form(None),
    image: Optional[UploadFile] = File(None),
    request: Request = None,
    current_user: UserInDB = Depends(get_current_active_user)
}

"""Generate jewelry budget recommendations with optional outfit image"""
# Create the budget input model
budget_input = JewelryBudgetInput(
    total_budget=total_budget,
    occasion=occasion,
    preferences=preferences
)

image_path = None
if image:
    image_path = save_upload_file(image)

# Store last budget planning in session data
if current_user.username in active_sessions:
    active_sessions[current_user.username].user_data["last_jewelry_budget"] = {
        "timestamp": datetime.datetime.now().isoformat(),
        "budget": budget_input.total_budget,
        "occasion": budget_input.occasion,
        "has_image": image is not None
    }

# Get recommendations
result = get_jewelry_recommendations(budget_input, image_path)

# Save to history with image info
input_data = budget_input.dict()
if image:
    input_data["image"] = image.filename

save_to_history(
    username=current_user.username,
    recommendation_type="jewelry",
    input_data=input_data,
    result=result
)

return result

@app.get("/recommendation-history")
def get_recommendation_history():
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
}

"""Get the user's recommendation history"""
if current_user.username not in user_recommendations:
```

```
# Sort history by timestamp (newest first)
history = sorted(
    user_recommendations[current_user.username],
    key=lambda x: x.timestamp,
    reverse=True
)

# Convert to dict for JSON response
history_data = []
for item in history:
    history_data.append({
        "id": item.id,
        "timestamp": item.timestamp,
        "type": item.recommendation_type,
        "input": item.input_summary,
        "summary": item.result_summary
    })

return {"history": history_data}
```

- User Auth (/register, /login, /logout, /token): Manages user registration, authentication, and session control.
- Session Info (/session-info, /session-data): Retrieves user-specific session data for personalized AI interaction.

• Activity 3.2: Modular Architecture Setup

/recommendations-details: Returns detailed AI-generated product recommendations based on user budget, preferences, and selected category (Home, Party, or Jewelry).

```
# Step 6: Add an endpoint to get details for a specific recommendation
@app.get("/recommendation-details/{recommendation_id}")
async def get_recommendation_details(
    recommendation_id: str,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Get the full details of a specific recommendation"""
    if current_user.username not in user_recommendations:
        raise HTTPException(status_code=404, detail="No recommendations found")

    # Find the recommendation with the given ID
    for item in user_recommendations[current_user.username]:
        if item.id == recommendation_id:
            return {
                "id": item.id,
                "timestamp": item.timestamp,
                "type": item.recommendation_type,
                "input": item.input_summary,
                "full_result": item.full_result
            }

    raise HTTPException(status_code=404, detail="Recommendation not found")
```

• Activity 3.3: CORS & Static Routing

```
# Step 7: Add a new endpoint to serve the history page
@app.get("/history", response_class=HTMLResponse)
async def history_page(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """History page to view past recommendations"""
    return templates.TemplateResponse("history.html", {"request": request, "user": current_user})
```

• Activity 3.4: Startup and main function

`/startup`: Initializes essential application services and loads configuration settings when the FastAPI server starts.

```
@app.on_event("startup")
async def setup_session_cleanup():
    """Background task to clean up expired sessions"""
    async def cleanup_expired_sessions():
        while True:
            current_time = datetime.utcnow()
            # Check for sessions that have been inactive for more than 30 minutes
            expired_sessions = [
                username for username, session in active_sessions.items()
                if (current_time - session.last_activity).total_seconds() > 1800 # 30 minutes
            ]

            # Remove expired sessions
            for username in expired_sessions:
                if username in active_sessions:
                    print(f"Removing expired session for {username}")
                    del active_sessions[username]

            # Wait for 5 minutes before checking again
            await asyncio.sleep(300)

    # Start the background task
    asyncio.create_task(cleanup_expired_sessions())
```

`__main__`: Entry point of the FastAPI application that starts the server using `uvicorn` when the script is run directly.

```
# Main entry point
if __name__ == "__main__":
    import uvicorn
    print("Starting PocketSmart: AI Budget Planner...")
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

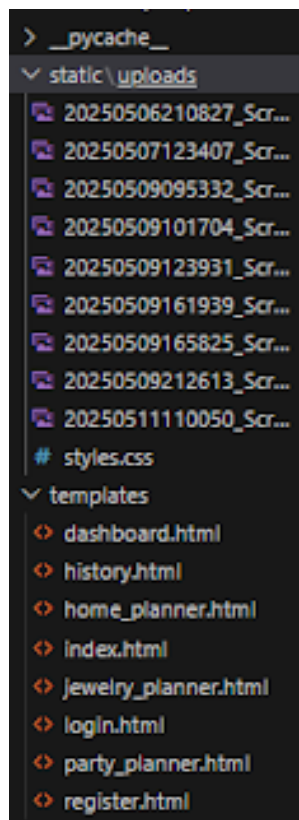
MILESTONE 4: UI Development

This milestone focuses on developing a **lightweight, responsive, and intuitive frontend** for **PocketSmart AI** using **HTML, CSS, and Jinja2 templates**. The interface enables users to interact seamlessly with the system's

Activity 4.1: Develop HTML Frontend Interface

- Design a home page with budget input fields and category selectors (Home, Party, Jewelry)
- Implement individual forms for each planner module connected to FastAPI POST routes
- Display AI responses in well-structured, card-like layouts (e.g., product suggestions, item details, prices)

Templates Directory Structure:



MILESTONE 5: Testing & Optimization

This milestone focuses on validating the reliability, accuracy, and contextual awareness of PocketSmart AI's recommendation engine. The system is tested using real-world user scenarios across different planning categories —Home Interiors, Party Planning, and Jewelry Suggestions—to ensure consistent performance. Optimization efforts include refining AI prompt structures, improving session handling, and ensuring a smooth UI/UX across devices.



- Home decor setups from platforms like IKEA, Amazon
- Party packages including food (Swiggy/Zomato), venues (OYO), and decor
- Jewelry suggestions matched with outfit styles, occasions, and image inputs

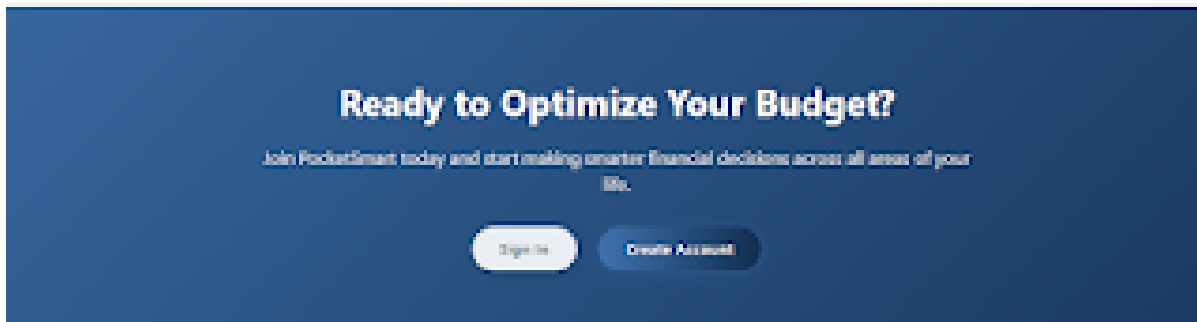
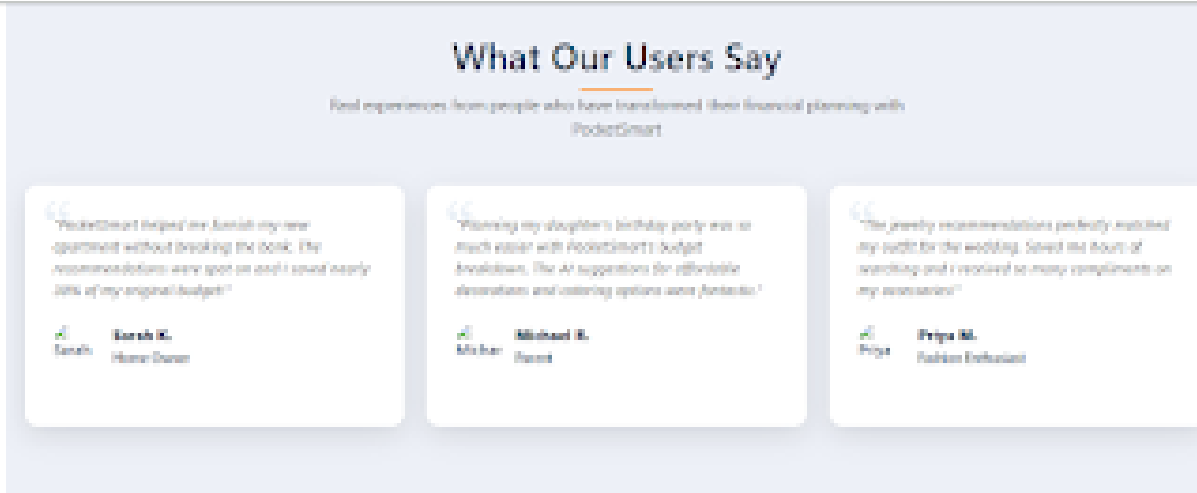
● Home Page

Home Page: The main landing page introducing PocketSmart AI’s features with links to planner modules and user actions.

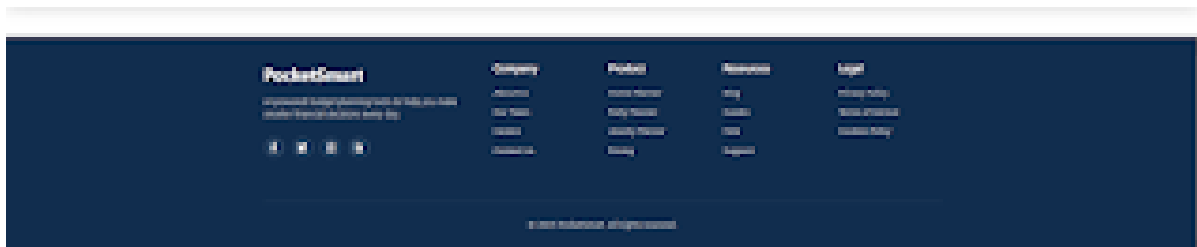


● Testimonials Page

Testimonials Page: Showcases real user reviews and success stories to demonstrate the platform’s effectiveness.



• Footer Part



• Register Page

Register Page: Allows new users to create an account to access personalized budget recommendations.

AI-Powered Budget Planning

Create Your Account

Username

 Choose a username

Email

 Enter your email

Password

 Create a strong password

Confirm Password

 Confirm your password

Create Account 

Already have an account? [Sign In](#)

- **Login Page**^[a]

Login Page: Authenticates existing users to access their dashboard and saved data.

POCKETSMART

AI-Powered Budget Planning

Welcome Back

Username

 Enter your username

Password

 Enter your password

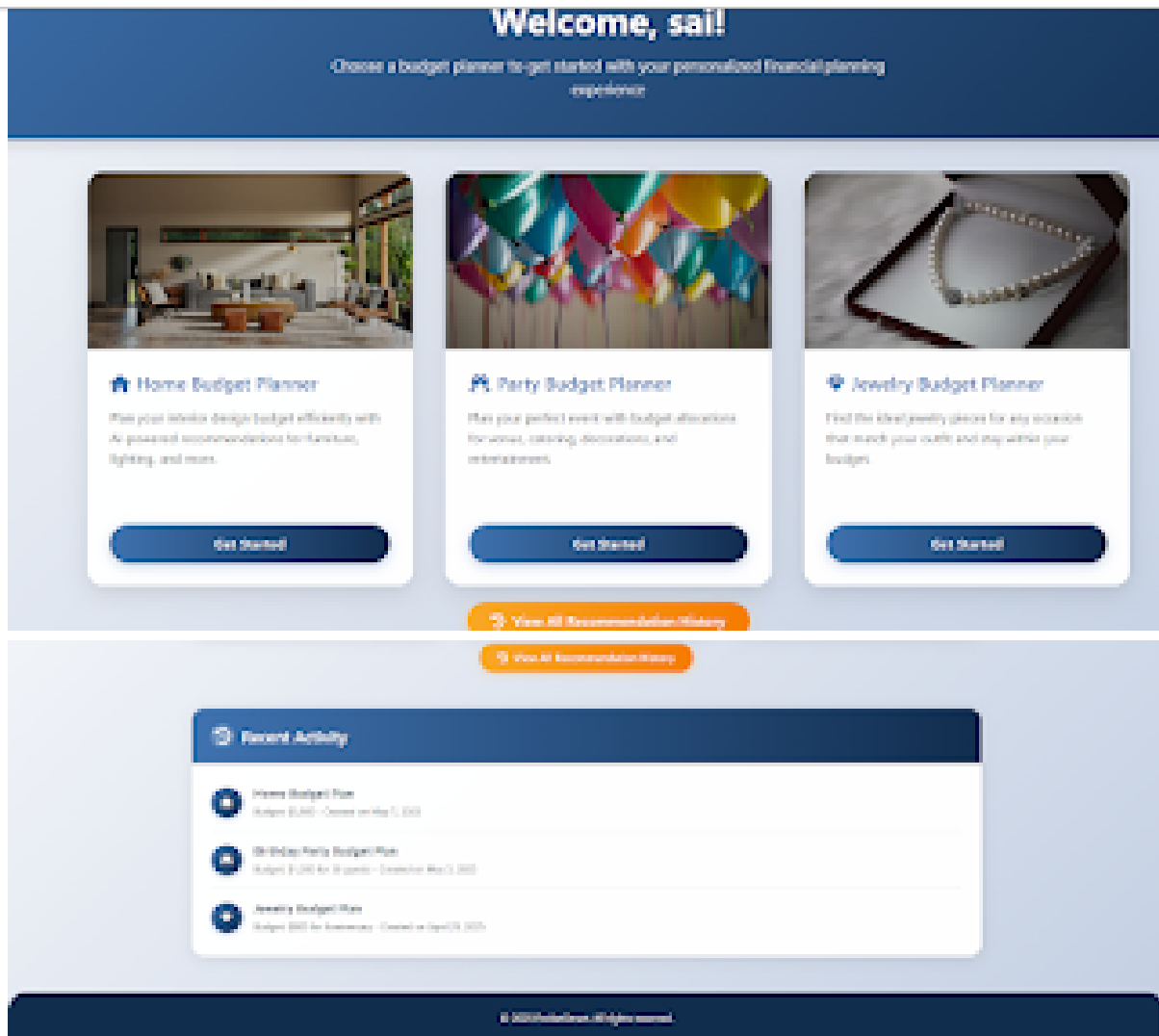
[Forgot Password?](#)

Sign In →

Don't have an account? [Create Account](#)

- **User Dashboard**

User Dashboard: Displays recent recommendations, saved queries, and personalized insights for the logged-in user.



Home Interior Budget Planner Page: Lets users input home decor preferences and budget to get tailored suggestions from multiple platforms.

Home Interior Budget Planner

Create a customized budget plan for your dream home interior

Budget Details

Total Budget (\$)

\$

5000

Pictures & Furniture

Number of Lights/Pictures

4

Number of Dining Pans

4

Number of Furniture/Pieces

4

Number of Dining Tables

1

Rooms to Include

☒ Living Room

☒ Kitchen

☐ Bedroom

Additional Information

Special Requirements or Preferences

Any specific requirements or preferences...

Generate Recommendations

Home Interior Recommendations Page: Displays AI-generated product suggestions for home decor based on the user's budget and style preferences.

Your Personalized Budget Plan

Budget Summary

Total Budget: \$1000.00

Remaining Budget: **\$145.00**

Lighting

Allocation: \$1000.00

Item	Description	Price	Quantity	Shopping Links
LED Bulbs (Warm White)	Energy-efficient LED bulbs for general lighting.	\$100.00	1	Amazon Walmart eBay Target iStock

Ceiling Fans

Allocation: \$2000.00

Item	Description	Price	Quantity	Shopping Links
Recessed Ceiling Fan	Basic functional ceiling fan.	\$500.00	4	Amazon Walmart eBay Target iStock

Furniture

Allocation: \$1000.00

Item	Description	Price	Quantity	Shopping Links
Plastic Chair	Durable plastic chairs for kitchen or living room.	\$200.00	2	Amazon Walmart eBay Target iStock
Small Wooden Table	Simple wooden table for dining or side table.	\$500.00	1	Amazon Walmart eBay Target iStock

Additional Suggestions

- Consider purchasing used furniture for further cost savings.
- Look for sales and discounts on online marketplaces.
- Prioritize essential items and postpone non-essential purchases.

Party Budget Planner Page: Helps users plan events by suggesting venues, food, and decorations based on budget and guest count

Party Budget Planner

Plan your perfect event with AI-powered budget recommendations

1 Basic Information

Total Budget (\$)

Number of Guests

\$500

10

2 Event Details

Party Type

Venue Type

Wedding

Home

3 Party Needs

Catering

Decoration

Entertainment

4 Additional Requirements

Special requests, themes, dietary restrictions, etc.

Generate Budget Plan

© 2023 PocketSmart. All rights reserved.

Party Budget Recommendations Page: Shows curated party planning options, including food, venues, and decorations, matched to the provided budget.

Jewelry Budget Planner

Get AI-powered jewelry recommendations within your budget for any occasion

Budget Details

Total Budget (\$):

\$

1000

Occasion & Preferences

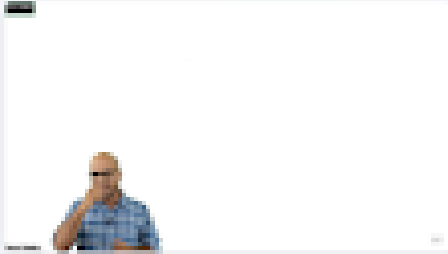
Occasion

Wedding

Style Preferences

Describe your style preferences, materials, colors, etc.

Upload Outfit Image

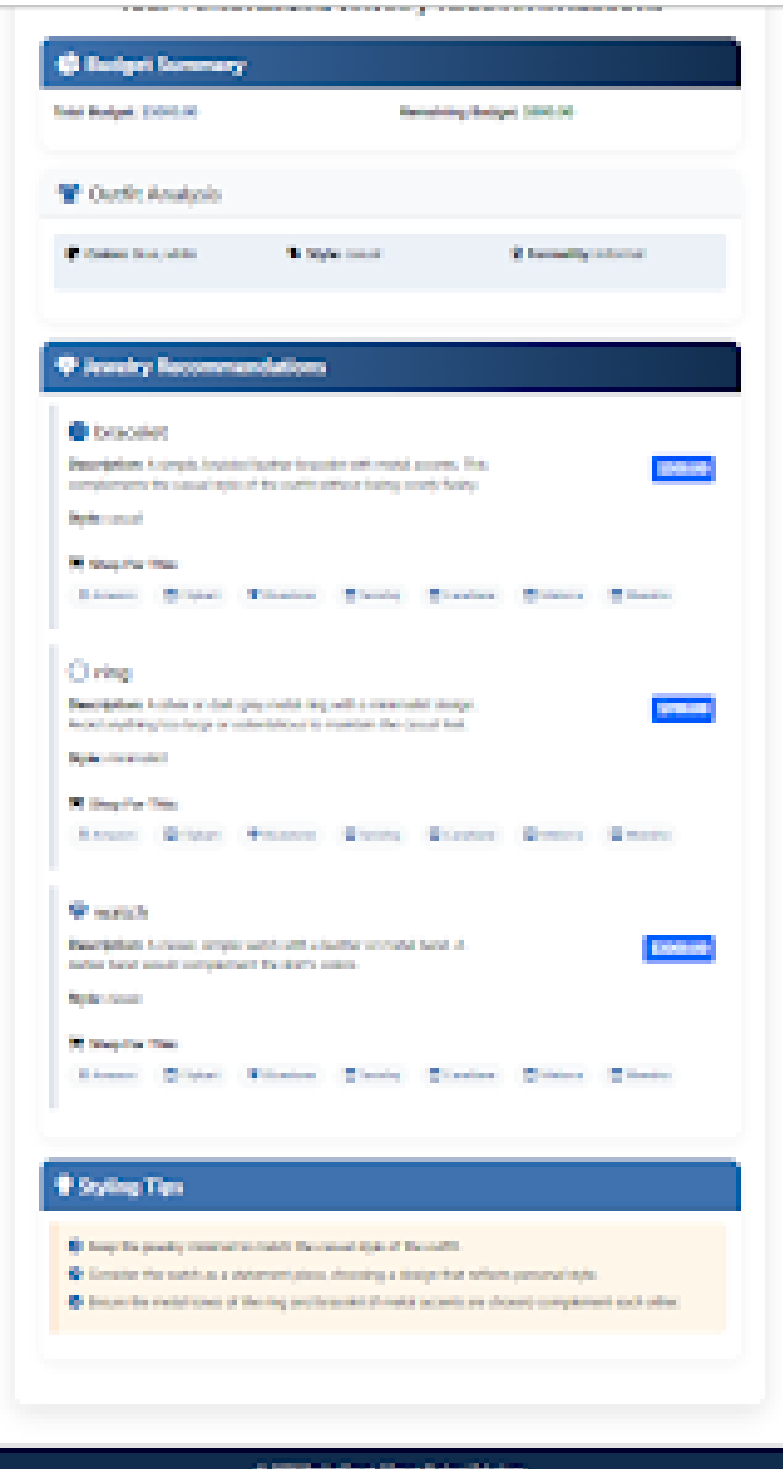


Remove Image

Get Recommendations

© 2024 PocketSmart | Smart Budget Solutions

Jewelry Budget Recommendations Page: Presents personalized jewelry options tailored to the occasion, outfit, and budget, using Gemini-generated insights.



Recommendation History Page: Displays a log of the user's past recommendation queries and results, allowing easy review and reuse of previous plans.



Your Recommendation History

View and manage all your previous budget plans and recommendations

Recent Recommendations

Jewelry Budget

May 21, 2021, 12:46 PM

Total Budget

₹5,000

Remaining

₹800

Occasion: Birthday

With audit image: Yes

View Full Details

Party Planning Budget

May 22, 2021, 10:00 PM

Total Budget

₹5,000

Remaining

₹0

Party Type: Wedding

Guests: 3

Needs: Catering, Entertainment

Decor

Catering

Entertainment

Decor

View Full Details

Home Interior Budget

May 22, 2021, 10:00 PM

Total Budget

₹5,000

Remaining

₹100

Rooms: Living Room, Kitchen

Lights: 3

Furniture: 4

Furniture: 2

Lighting

Cooking Area

Furniture

View Full Details

aq

Conclusion

PocketSmart AI redefines the way individuals plan budgets for everyday lifestyle needs by delivering smart, AI-driven recommendations across **home interiors, party planning, and jewelry selection**. By combining the power of **FastAPI** with **Gemini 1.5 Flash Pro**, the system intelligently processes user inputs—budgets, preferences, and even images—to generate accurate and context-aware suggestions from multiple trusted platforms like **Amazon, Flipkart, IKEA, Swiggy, Zomato, OYO and more**. From backend architecture to AI integration and a responsive frontend, PocketSmart AI has been built with a focus on **efficiency, scalability, and user-centric design**. The FastAPI backend ensures modular, secure routing and robust session handling, while the Jinja2-powered frontend offers an intuitive and clean interface for users to interact with the system in real time.

Each module—from registration to recommendations—is designed to simplify decision-making and enhance the shopping or planning experience through automation and personalization. Whether it's finding the right wall art within budget or organizing a last-minute party, PocketSmart AI acts as a **smart digital assistant**, saving time and effort while optimizing outcomes. With its seamless fusion of generative AI and web technologies, **PocketSmart AI** stands as a practical example of how **AI can enhance everyday decision-making**, bringing intelligent recommendations to users' fingertips—budget-friendly, fast, and personalized.

[\[a\]http://console.cloud.google.com](http://console.cloud.google.com)